Valiant Verification SDK

The plain-English version first, the machinery after.

The idea in 30 seconds

Your tests pass every day. That tells you almost nothing. A test that always passes is worthless — but CI shows it green, and you ship on it. The only way to know a test is real is to see it *fail* when the thing it protects breaks. Almost nobody checks that.

VVS checks it for you. It **breaks your code on purpose** — flips a `+` to a `-`, deletes a function, inverts a condition — and watches: **did your test notice?**

- Your test went red → the test is **real**. Green finally means something.
- Your test stayed green → the test is **lying**, and VVS refuses to call it verified.

It also closes the two other gaps CI lets slide: a test that *didn't run* is a fail, not a pass; and you can't ship while any of this is red.

> **The analogy — a fire drill for your tests.** You don't trust a fire alarm until you've set off smoke and heard it ring. VVS sets off smoke in your code and checks your tests ring. An alarm that doesn't ring on real smoke isn't an alarm.

A 20-second concrete example

```
// your code
function sum(a, b) { return a + b; }

// your test
assert(sum(2, 3) === 5);
```

`valiant mutate` rewrites `a + b` into `a - b`. Your test must now fail — and it does, so VVS stamps it **CAUGHT** (real). Had the test still passed, VVS would stamp it **SURVIVED** (lying) and refuse to call it a check.

That is the entire product. It does not know what `sum` should do — *you* told it, in your test. It only proves your test would notice if that broke. Hence the honest boundary:

> **VVS verifies your verification, not your code.** It's a calibrated scale, not a chef: it makes the measurement honest and proves the scale isn't lying. How much of your code the tests cover is still your job.

What it guarantees (and what it doesn't)

IT *DOES* GUARANTEE	IT DOES *NOT* GUARANTEE
the tests ran (a skipped test is a fail)	the code is correct for every input — that's undecidable
each test was shown to fail on a real breakage	you wrote *enough* tests — coverage is yours
the result is a number, not an adjective	you tested the *right* things

the record is tamper-evident

How it actually does it

1 · The gate — a check returns a number

Every check returns an exit code: `0` holds, `1` broken, `2` could-not-run. The key one is `2`: CI treats “no output / skipped” as a pass; VVS treats it as a *fail*.

2 · Mutation — prove the check can fail

It sabotages the code (a deterministic find/replace), then requires the check to go red. The verdicts: **CAUGHT** (real), **SURVIVED** (passed with the subject broken — lying), **STALE** (sabotage matched nothing — no-op), **ALWAYS-RED** (fails with nothing sabotaged), **CANNOT-RUN** (couldn’t run).

3 · Enforcement — block, don’t report

A hook runs the gate when an agent tries to *finish* (claim completion, e.g. `git commit/push`). A non-zero exit blocks it and hands it the reason. `valiant mcp` exposes the same verdicts as structured data so an agent can ask.

4 · Self-check — the SDK verifies itself first

`SELF.json` is a sha256 manifest of every shipped file; before verifying your repo, the SDK verifies itself, so a tampered install can’t produce a green.

Exactly what it does, in code

The config — what a gate actually IS

```
{
  "gates": [
    {
      "name": "selftest-licence",
      "cmd": ["node", "tools/selftest-licence.mjs"],
      "subject": "src/licence.mjs",
      "sabotage": { "find": "export function verifyLicence",
                    "replace": "export function verifyLicenceX",
                    "why": "signature verification is renamed away" }
    }
  ]
}
```

A gate is exactly three things: a **command** (`cmd`, an argv array), a **subject** (the file it protects), and a **sabotage** (a `find`/`replace` on that file). That is the entire data model — there is no other config.

`valiant gate` — spawn the command, then judge the output

```
function runGate(gate, repoRoot) {
  const { name, cmd } = gate;
  const res = runCheck(cmd, cmd.slice(1), { cwd: repoRoot });  // spawnSync: {status, stdout, stderr}
  if (res.error)  return { name, ok: false, detail: "did not run" };
  if (res.status === 2) return { name, ok: false, cannotCheck: true,
                                 detail: "cannot check (exit 2) — never a silent pass" };
  if (res.status !== 0) return { name, ok: false, detail: `exit ${res.status}` };
  // exit 0 is NOT enough — a pass must produce evidence:
  if (!out && !err) return { name, ok: false,
                             detail: "exit 0 with no output — suspect" };
  return { name, ok: true, detail: "exit 0" };
}
```

The key lines: it spawns your command, reads the exit code, and **rejects exit 0 with no output** — a test that prints nothing and passes is a fail, not a pass. Exit `2` is “could not run” and is also a fail.

`valiant mutate` — the sabotage loop, verbatim

```
function runMutation(guard, repoRoot) {
  const { file, find, replace, check } = guard;
  // 1. copy the repo to a throwaway dir — the real repo is never touched
  const count = src.split(find).length - 1;
  if (count === 0) return { result: "stale", detail: "find not present (0 matches)" };
  if (count  > 1) return { result: "stale", detail: `find matches ${count} times (must be exactly once)`
  // 2. run the check on the CLEAN copy first:
  if (first.status !== 0) return { result: "always-red", detail: "red with NOTHING sabotaged" };
  // 3. apply the sabotage (buffer indexOf + concat, NOT String.replace):
  const i = src.indexOf(find);
  const sabotaged = Buffer.concat([src.subarray(0, i), Buffer.from(replace), src.subarray(i + find.length
  // 4. run the check under the sabotage:
  if (under.status === 0) return { result: "survived", detail: "PASSED with the sabotage applied" };
  // 5. undo and re-run:
  if (restored.status !== 0) return { result: "unstable", detail: "stayed red after undo" };
  return { result: "caught", detail: "went red under sabotage, green once undone" };
}
```

Every verdict is a *code path*, not an adjective: `stale` = the find matched 0 or 2+ sites; `always-red` = red with nothing broken; `survived` = green with the sabotage applied; `unstable` = stayed red after undo; `caught` = red under sabotage and green restored. It uses a `Buffer` edit (not `String.replace`) so the sabotage is byte-exact and the undo can never corrupt the file.

Self-check — `SELF.json`

```
SELF.json = { "bin/valiant.mjs": "", "src/gates.mjs": "", ... }
```

Every shipped file is hashed. Before verifying your repo, the SDK re-hashes its own files and compares — any mismatch fails the `selfcheck` gate, so a tampered or half-updated install cannot produce a green.

The hook — what actually blocks the agent

A PreToolUse/Stop hook is a shell one-liner that runs `valiant gate --hook`. That reads the event the host passed on stdin, extracts the command the agent is about to run, and only gates *completion* commands (`git commit/push`, `gh pr create/merge`). Non-zero exit = **block**, with the reason on stderr. Everything else passes straight through, so the gate blocks the moment it matters instead of every tool call.

A real walkthrough

```
curl -fsSL https://vvs.morpheus.nz/install.sh | sh   # install, opens the control panel
valiant setup                                        # writes valiant.json + proposes a check
valiant mutate                                       # breaks the code → the check must go red
valiant gate                                         # 0/1/2 → what CI + hooks enforce
valiant verify                                       # your checks + shipped checks + self-check
valiant control                                      # 0/1/2 — did the AI stay under human control
```

`valiant.json` is the one config file it adds — it names the checks and the enforcement stance. That's the whole surface: one file, one exit-code contract, and a proof that every check can fail.

APPENDIX · THE SHIPPED RULEBOOK (THE CORPUS)

The SDK ships battle-tested rules, each traceable to a real incident. You don't have to read any of this to use it — it runs against your repo and flags matches.

COMPONENT	COUNT	WHAT IT IS
Universal invariants	46	the rules every claim must hold
Hazards that bought them	56	the real failures each rule prevents
Operating-method rules	24	the discipline: memory, verify, build order, reporting
Reference guards	29	the code that enforces the rules

The 46 invariants

1 Contract integrity · **2** Silent no-op edits forbidden · **3** Bound the model's output · **4** Schema ships with its migration · **5** Sync against the live source · **6** Verify before depending · **7** Schema drift degrades gracefully · **8** A module is verified only when loaded · **9** A silent zero is a bug · **10** Never neutralise a build step · **11** Review behaviour, not the export surface · **12** Data isolation is a decision · **13** "Nothing failed" ≠ "the gates ran" · **14** A declaration outlives its block · **15** Prove the check can fail · **16** Name the machines it runs on · **17** The verdict is the build's · **18** Lint coverage, render the real surface · **19** A tool that rewrites the tree owns it · **20** Close stdin on subprocesses · **21** Portable filenames · **22** The loop survives its queue · **23** A refresh preserves what it can't derive · **24** A tool that can't do its job exits non-zero · **25** Liveness is proved, not inferred · **26** No faster-than-interval repeats · **27** Evidence recomputed, never transcribed · **28** Stalled ≠ slow · **29** A pipeline must not mask its status · **30** Edit a generated file at its generator · **31** A parser isn't a type-checker · **32** Never edit what you haven't read · **33** Never edit what you don't understand · **34** A claim of inability is checked, not announced · **35** Measuring under load measures the rig · **36** Stable context first · **37** Search before building · **38** A missing capability is a task · **39** Blast radius before action · **40** A UI change is verified by looking · **41** No dead ends · **42** Evidence reaches the verifier · **43** A wait loop attempts the repair · **44** AI action approved · **45** AI claim traceable · **46** AI intent restated.

The 24 method rules

M1 Write it down or it is lost · **M2** The injected file must stay small · **M3** Record the incident with the rule · **M4** Handover is pull, not push · **M5** A stale handover is worse than none · **M6** Never claim, always measure · **M7** Prove the check can fail · **M8** Distinguish cannot-check from pass · **M9** Ask the gate, don't parse its source · **M10** A green local check says nothing about the other list · **M11** Smallest change that can be verified · **M12** Root-cause before patching · **M13** Fix the generator, not its output · **M14** After every incident, write the rule AND the guard · **M15** One workspace per worker · **M16** A seconds-fast failure is a missing name, not a slow test · **M17** Say what is NOT done · **M18** Evidence travels with the claim · **M19** Never wait on a queue you could bypass · **M20** Do the next authorised step · **M21** Turn every reminder into a rule or a gate · **M22** Probe before declaring a wall · **M23** Check the blast radius before acting · **M24** Pre-authorise the reversible.

The 56 hazards (the real incidents behind the rules)

A shared module loses an export. An admin query over-filters and hides system rows. A generated file edited by hand, lost on rebuild. A lockfile not committed, rewritten on install. An edit matching zero/multiple sites ships nothing. An unbounded model call truncated by a provider default. A push wiping protected paths. A schema change without its migration. A push diffing against a stale snapshot. Code written against an API shape from memory. A new column breaking a feature until SQL lands. A wrong import passing gates, killing the container at boot. An empty query forever with nothing saying so. A green build shipping an artifact that can't run. A module losing behaviour but keeping its exports. An admin read exposing another account's data. A check that never ran reading as passed. A `try` hiding declarations from its catch. A guard satisfied by the bug it exists to prevent. A build running only on its own machine. A failed save reading as a failed build. A lint-blocked file checked by no rules. A tree-rewriter restoring a stale backup. A subprocess prompting on inherited stdin. A filename Windows can't represent. A loop killable by its own queue. A refresh dropping hand-added fields. A repair tool reporting a sick node healthy. A board reading a dead node green from cache. A restart script restarting a service every 41s. A badge counting hand-transcribed evidence. A wedged loop reading as slow. A drift check silently removed. Operating without reading the rules. Instructions not matching the system. A stale record trusted over the live machine. A live-check claim with no live check. Hedging on a finished delivery. Work finished with no handoff. A rule ported without validation. A job printing PASS while both checks failed. A live config overwritten by the installer. Type errors reaching a gate because a formatter parsed the file. An edit applied to an unread file. A flake "found" on a compiling machine. An instruction file grown until it dropped the safety rules. Two sessions building the same thing. A missing tool reported as a wall. A local change taking down a shared capability. A UI change merged unlooked-at. An error state with no next step. A guard edited in the deployed copy, fix vanishing. Code changed to hide a symptom the author couldn't explain. An abandoned download in-flight for five days. A flashed image refusing its key, rebuilt five times. Sleep-polling a queued CI job ~30 times with five idle nodes.

The 29 reference guards (code → rule)

GUARD	INVARIANT
`contract-integrity`	1 — contract integrity

`resolve-imports`	8 — a module is verified when loaded
`model-call-budget`	3 — bound the model's output
`convention-queries-bounded`	9 + 7 — silent zero / schema drift
`no-swallowed-build`	10 — never neutralise a build step
`tenant-predicate`	12 — data isolation
`declaration-scope`	14 — a declaration outlives its block
`ci-can-skip`	17 — a skipped check isn't a pass
`lint-coverage`	18 — lint coverage
`closed-stdin`	20 — close stdin on subprocesses
`portable-filenames`	21 — portable filenames
`convention-loop-survives`	22 — the loop survives its queue
`convention-refresh-preserves`	23 — a refresh preserves
`nonzero-on-failure`	24 — exit non-zero on failure
`convention-status-proved`	25 — liveness proved
`timers-that-loop`	26 — no faster-than-interval repeats
`progress-visible`	28 — stalled vs slow
`pipeline-masks-status`	29 — no masked status
`generated-file`	30 — edit at the generator
`typecheck-is-real`	31 — a parser isn't a type-checker
`convention-measure-load`	35 — measure the rig, not the code
`convention-context-order`	36 — stable first, volatile last
`convention-ui-screenshot`	40 — UI verified by looking
`convention-errors-actionable`	41 — no dead ends
`evidence-reaches-verifier`	42 — evidence reaches the verifier
`wait-must-try-to-fix`	43 — a wait loop attempts the repair
`ai-control-approved`	44 — AI action approved
`ai-control-traceable`	45 — AI claim traceable
`ai-control-restated`	46 — AI intent restated

APPENDIX · THE LICENCE

The licence is an ed25519-signed envelope — `vvs1.<payload>.<signature>` — carrying plan, seats, and expiry. It verifies **offline** against a shipped public key: no call home. Revocation is a signed list the SDK can pull. The private key never leaves the issuer's machine.